Practice Session on Queues

What is a Queue?

A queue is a linear data structure that follows the **First In, First Out (FIFO)** principle. Elements are added at the rear (back) and removed from the front. Think of it like a line of people waiting - the first person in line is the first to be served.

Queue Operations

Basic Queue Operations:

- **Enqueue**: Add an element to the rear of the queue
- **Dequeue**: Remove an element from the front of the queue
- **Front/Peek**: View the front element without removing it
- **isEmpty**: Check if the queue is empty
- **Size**: Get the number of elements in the queue

Creating Queues in Java

Method 1: Using LinkedList Class

```java
import java.util.LinkedList;
import java.util.Queue;

public class QueueExample {
    public static void main(String[] args) {
        Queue<Integer> queue = new LinkedList<>();

        // Adding elements (enqueue)
        queue.offer(10);
        queue.offer(20);
        queue.offer(30);

        System.out.println("Queue: " + queue);

        // Removing elements (dequeue)
        int removed = queue.poll();
        System.out.println("Removed: " + removed);
        System.out.println("Queue after removal: " + queue);

        // Peek at front element
        int front = queue.peek();
        System.out.println("Front element: " + front);
    }
}
```

Method 2: Using ArrayDeque Class

```java
import java.util.ArrayDeque;
import java.util.Queue;

public class ArrayDequeExample {
    public static void main(String[] args) {
        Queue<String> queue = new ArrayDeque<>();

        // Adding elements
        queue.add("First");
        queue.add("Second");
        queue.add("Third");

        System.out.println("Queue: " + queue);

        // Processing elements
        while (!queue.isEmpty()) {
            System.out.println("Processing: " + queue.remove());
        }
    }
}
```

Method 3: Using Priority Queue

```java
import java.util.PriorityQueue;

public class PriorityQueueExample {
    public static void main(String[] args) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();

        // Adding elements (they will be ordered automatically)
        pq.offer(30);
        pq.offer(10);
        pq.offer(20);

        System.out.println("Priority Queue: " + pq);

        // Elements come out in sorted order
        while (!pq.isEmpty()) {
            System.out.println("Removed: " + pq.poll());
        }
    }
}
```

Important Queue Methods

Method	Description	Returns
`offer(element)`	Adds element to rear	boolean
`add(element)`	Adds element to rear	boolean
`poll()`	Removes and returns front element	Element or null
`remove()`	Removes and returns front element	Element or Exception
`peek()`	Returns front element without removing	Element or null
`element()`	Returns front element without removing	Element or Exception
`isEmpty()`	Checks if queue is empty	boolean
`size()`	Returns number of elements	int

Practice Questions on Queue Creation

Question 1: Basic Queue Operations

```java
import java.util.LinkedList;
import java.util.Queue;

public class BasicQueueOps {
    public static void main(String[] args) {
        Queue<Integer> q = new LinkedList<>();

        // Add numbers 1 to 5
        for (int i = 1; i <= 5; i++) {
            q.offer(i);
        }

        // Remove first two elements
        q.poll();
        q.poll();

        // Add 6 and 7
        q.offer(6);
        q.offer(7);

        // Print remaining elements
        System.out.println("Final queue: " + q);
        // Output: Final queue: [3, 4, 5, 6, 7]
    }
}
```

Question 2: Queue with Strings

```java
import java.util.ArrayDeque;
import java.util.Queue;

public class StringQueue {
    public static void main(String[] args) {
        Queue<String> names = new ArrayDeque<>();

        names.add("Alice");
        names.add("Bob");
        names.add("Charlie");

        System.out.println("First person: " + names.peek());
        System.out.println("Serving: " + names.poll());
        System.out.println("Next person: " + names.peek());

        // Output:
        // First person: Alice
        // Serving: Alice
        // Next person: Bob
    }
}
```

Question 3: Check Queue Status

```java
import java.util.LinkedList;
import java.util.Queue;

public class QueueStatus {
    public static void main(String[] args) {
        Queue<Double> scores = new LinkedList<>();

        System.out.println("Is empty: " + scores.isEmpty());

        scores.offer(85.5);
        scores.offer(92.0);
        scores.offer(78.5);

        System.out.println("Size: " + scores.size());
        System.out.println("Is empty: " + scores.isEmpty());

        // Output:
        // Is empty: true
        // Size: 3
        // Is empty: false
    }
}
```

Question 4: Processing All Elements

```java
import java.util.LinkedList;
import java.util.Queue;

public class ProcessQueue {
    public static void main(String[] args) {
        Queue<String> tasks = new LinkedList<>();

        tasks.offer("Task A");
        tasks.offer("Task B");
        tasks.offer("Task C");

        System.out.println("Processing tasks:");
        while (!tasks.isEmpty()) {
            String currentTask = tasks.poll();
            System.out.println("Completed: " + currentTask);
        }

        System.out.println("All tasks completed!");
    }
}
```

Question 5: Queue Size Management

```java
import java.util.ArrayDeque;
import java.util.Queue;

public class QueueSizeDemo {
    public static void main(String[] args) {
        Queue<Integer> numbers = new ArrayDeque<>();

        // Add elements until size is 4
        for (int i = 10; i <= 40; i += 10) {
            numbers.offer(i);
            System.out.println("Added " + i + ", Size: " + numbers.size());
        }

        // Remove elements one by one
        while (!numbers.isEmpty()) {
            int removed = numbers.poll();
            System.out.println("Removed " + removed + ", Size: " + numbers.size());
        }
    }
}
```

Quiz Questions

1. What principle does a Queue follow? a) Last In, First Out (LIFO) b) First In, First Out (FIFO) c) Random Access d) Priority Based

Answer: b) First In, First Out (FIFO)

2. Which method is used to add an element to a queue? a) push() b) insert() c) offer() d) append()

Answer: c) offer()

3. What does the poll() method return when the queue is empty? a) Throws an exception b) Returns -1 c) Returns null d) Returns false

Answer: c) Returns null

4. Which class can be used to implement a Queue in Java? a) ArrayList b) LinkedList c) Vector d) Stack

Answer: b) LinkedList

5. What is the difference between peek() and poll() methods? a) peek() removes the element, poll() doesn't b) poll() removes the element, peek() doesn't c) Both remove the element d) Neither removes the element

Answer: b) poll() removes the element, peek() doesn't

Practice Coding Questions

1. Customer Service Queue Write a Java program that simulates a customer service queue. Create a queue of customer names, add 5 customers, then serve them one by one while displaying who is being served and how many customers are left in the queue.

2. Number Reversal Using Queue Create a program that takes integers from 1 to 10, adds them to a queue, then removes them and adds them to another queue in reverse order. Display both queues.

3. Queue Operations Menu Write a program that provides a menu-driven interface for queue operations:

- Add element
- Remove element
- Display front element
- Display queue size
- Display all elements
- Exit

4. Task Priority Queue Create a program using PriorityQueue to manage tasks with different priorities (1-5, where 1 is highest priority). Add 8 tasks with different priorities and then process them in priority order.

5. Queue Comparison Write a program that creates two identical queues with the same elements. Implement a method to compare if two queues contain the same elements in the same order without modifying the original queues.